new idea

WINDOWS

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

For each teacher, give average salary of his department.

```
SELECT name,
    (SELECT avg(salary)
     FROM teacher AS B
     WHERE B.dept_name = A.dept_name)

FROM teacher AS A;
```

name	*avg*
Srinivasan	77333.333333333333
Wu	85000.000000000000
Mozart	40000.000000000000
Einstein	91000.000000000000
El Said	61000.000000000000
Gold	91000.000000000000
Katz	77333.333333333333
Califieri	61000.000000000000
Singh	85000.000000000000
Crick	72000.000000000000
Brandt	77333.333333333333
Kim	80000.000000000000

For each teacher, give average of two top salaries of their department

```
WITH mytable(id, dept_name, salary) AS
   (SELECT A.id, A.dept_name, A.salary
    FROM teacher AS A
    WHERE 2 >= (SELECT count(*)
                FROM teacher AS B
                WHERE B.dept_name = A.dept_name
                AND B.salary >= A.salary)
   )

SELECT A.name, (SELECT avg(mytable.salary)
                FROM mytable
                WHERE A.dept_name = mytable.dept_name)
FROM teacher AS A;
```

For each teacher, give the average of salary just above, and just below his own (w.r.t. sorted salary order).

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

For each teacher, give the average of salary of the next 3 salaries above his own salary, and 2 previous salaries just below his own salary (w.r.t. sorted salary order).

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

Using nested subqueries can quickly become *very* complicated!

Some things we'd like to do in an easier way:

Use ORDER BY not only for output table, but to use **inside** a query

Use GROUP BY to calculate over groups but **don't collapse each group to one line**

Use AGGREGATIONS on groups of rows without grouping them

Main bottleneck: a table is an unordered set of tuples

a window : a set of rows that are ordered

Windows

1. Do SQL query as before, to get output table

2. Define a window for **each row of output table**

a window is an **ordered** subset of rows of the **output table**

3. In the **SELECT** statement, for each row, do aggregations on **its window rows**

IMPORTANT: The rows inside a window are ordered. It is **not** just a set or table

For each teacher, give the average of his salary, next higher salary, and next lower salary.

```
WINDOW w AS (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
```

+

```
SELECT name, salary, AVG(salary)
FROM teacher
```

↓

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

For each teacher, give the average of his salary, next higher salary, and next lower salary.

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

teacher

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

OVER is mandatory

name	*salary*	*avg*
Mozart	40000.00	50000.000000000000
El Said	60000.00	54000.000000000000
Califieri	62000.00	62333.333333333333
Srinivasan	65000.00	66333.333333333333
Crick	72000.00	70666.666666666667
Katz	75000.00	75666.666666666667
Singh	80000.00	78333.333333333333
Kim	80000.00	82333.333333333333
Gold	87000.00	85666.666666666667
Wu	90000.00	89666.666666666667
Brandt	92000.00	92333.333333333333
Einstein	95000.00	93500.000000000000

For each teacher, give the average of his salary, next higher salary, and next lower salary.

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

We create a window for **each row**

name	*salary*	*avg*
Mozart	40000.00	50000.000000000000
El Said	60000.00	54000.000000000000
Califieri	62000.00	62333.333333333333
Srinivasan	65000.00	66333.333333333333
Crick	72000.00	70666.666666666667
Katz	75000.00	75666.666666666667
Singh	80000.00	78333.333333333333
Kim	80000.00	82333.333333333333
Gold	87000.00	85666.666666666667
Wu	90000.00	89666.666666666667
Brandt	92000.00	92333.333333333333
Einstein	95000.00	93500.000000000000

① **ORDER BY salary**

for *each row* i: add to its window **all** rows of the output table sorted by salary

② **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING**

for *each row* i: only keep the rows $i-1$, i and $i+1$ in its window.

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

name	*salary*	*avg*
Mozart	40000.00	50000.000000000000
El Said	60000.00	54000.000000000000
Califieri	62000.00	62333.333333333333
Srinivasan	65000.00	66333.333333333333
Crick	72000.00	70666.666666666667
Katz	75000.00	75666.666666666667
Singh	80000.00	78333.333333333333
Kim	80000.00	82333.333333333333
Gold	87000.00	85666.666666666667
Wu	90000.00	89666.666666666667
Brandt	92000.00	92333.333333333333
Einstein	95000.00	93500.000000000000

```
SELECT name, salary, AVG(salary) OVER (ORDER BY SALARY
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM teacher;
```

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY);
```

name	*salary*	*avg*
Mozart	40000.00	40000.000000000000
El Said	60000.00	50000.000000000000
Califieri	62000.00	54000.000000000000
Srinivasan	65000.00	56750.000000000000
Crick	72000.00	59800.000000000000
Katz	75000.00	62333.333333333333
Singh	80000.00	66750.000000000000
Kim	80000.00	66750.000000000000
Gold	87000.00	69000.000000000000
Wu	90000.00	71100.000000000000
Brandt	92000.00	73000.000000000000
Einstein	95000.00	74833.333333333333

Default when ROWS not specified:

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
(ties included)

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW);
```

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY SALARY
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```



name	*salary*	*avg*
Mozart	40000.00	74833.333333333333
El Said	60000.00	74833.333333333333
Califieri	62000.00	74833.333333333333
Srinivasan	65000.00	74833.333333333333
Crick	72000.00	74833.333333333333
Katz	75000.00	74833.333333333333
Singh	80000.00	74833.333333333333
Kim	80000.00	74833.333333333333
Gold	87000.00	74833.333333333333
Wu	90000.00	74833.333333333333
Brandt	92000.00	74833.333333333333
Einstein	95000.00	74833.333333333333

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WHERE name LIKE '%an%'
WINDOW w AS (ORDER BY SALARY
       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```



name	*salary*	*avg*
Srinivasan	65000.00	78500.000000000000
Brandt	92000.00	78500.000000000000

Important: Windows are subsets of rows of the output table, not input table

For each teacher, give average of his salary and the next higher salary but only from **people in his department**.

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (PARTITION BY dept_name
        ORDER BY salary
        ROWS BETWEEN 0 PRECEDING AND 1 FOLLOWING);
```

teacher

id	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00
76543	Singh	Finance	80000.00
12121	Wu	Finance	90000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
15151	Mozart	Music	40000.00
33456	Gold	Physics	87000.00
22222	Einstein	Physics	95000.00

name	*salary*	*avg*
Mozart	40000.00	40000.000000000000
El Said	60000.00	61000.000000000000
Califieri	62000.00	62000.000000000000
Srinivasan	65000.00	70000.000000000000
Crick	72000.00	72000.000000000000
Katz	75000.00	83500.000000000000
Kim	80000.00	80000.000000000000
Singh	80000.00	85000.000000000000
Gold	87000.00	91000.000000000000
Wu	90000.00	90000.000000000000
Brandt	92000.00	92000.000000000000
Einstein	95000.00	95000.000000000000

For each teacher, give average of his salary and the next higher salary but only from **people in his department**.

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (PARTITION BY dept_name
        ORDER BY salary
        ROWS BETWEEN 0 PRECEDING AND 1 FOLLOWING);
```

① PARTITION BY dept_name

for *each row* i: add to its window all rows of the output table **with the same** `dept_name`

② ROWS BETWEEN 0 PRECEDING AND 1 FOLLOWING

for *each row* i: only keep the rows i and $i+1$ in its window.

name	*salary*	*avg*
Mozart	40000.00	40000.000000000000
El Said	60000.00	61000.000000000000
Califieri	62000.00	62000.000000000000
Srinivasan	65000.00	70000.000000000000
Crick	72000.00	72000.000000000000
Katz	75000.00	83500.000000000000
Kim	80000.00	80000.000000000000
Singh	80000.00	85000.000000000000
Gold	87000.00	91000.000000000000
Wu	90000.00	90000.000000000000
Brandt	92000.00	92000.000000000000
Einstein	95000.00	95000.000000000000

```
SELECT name, dept_name, salary,
       AVG(salary) OVER w as avg_dept_salary,
       MAX(salary) OVER w as max_dept_salary
FROM teacher
WINDOW w AS (PARTITION BY dept_name)
ORDER BY dept_name, salary;
```

This ORDER BY has nothing to do with window w
This is just the usual output sorting!

name	*dept_name*	*salary*	*avg_dept_salary*	*max_dept_salary*
Crick	Biology	72000.00	72000.000000000000	72000.00
Srinivasan	Comp. Sci.	65000.00	77333.333333333333	92000.00
Katz	Comp. Sci.	75000.00	77333.333333333333	92000.00
Brandt	Comp. Sci.	92000.00	77333.333333333333	92000.00
Kim	Elec. Eng.	80000.00	80000.000000000000	80000.00
Singh	Finance	80000.00	85000.000000000000	90000.00
Wu	Finance	90000.00	85000.000000000000	90000.00
El Said	History	60000.00	61000.000000000000	62000.00
Califieri	History	62000.00	61000.000000000000	62000.00
Mozart	Music	40000.00	40000.000000000000	40000.00
Gold	Physics	87000.00	91000.000000000000	95000.00
Einstein	Physics	95000.00	91000.000000000000	95000.00

```
SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER w
FROM teacher
GROUP BY dept_name
WINDOW w AS (ORDER BY dept_name
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

① Compute the output table rows

since we have GROUP BY, one row for each dept_name

dept_name	*avg*	*?column?*
Biology	72000.000000000000	
Comp. Sci.	77333.333333333333	
Elec. Eng.	80000.000000000000	
Finance	85000.000000000000	
History	61000.000000000000	
Music	40000.000000000000	
Physics	91000.000000000000	

```
SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER w
FROM teacher
GROUP BY dept_name
WINDOW w AS (ORDER BY dept_name
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

② **window: ORDER BY dept_name**

for *each row* i: add to its window all rows of the output table sorted by `dept_name`

③ **window: ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING**

for *each row* i: only keep the rows $i-1$, i and $i+1$ in its window.

dept_name	*avg*	*avg*
Biology	72000.000000000000	74666.666666666667
Comp. Sci.	77333.333333333333	76444.444444444444
Elec. Eng.	80000.000000000000	80777.777777777778
Finance	85000.000000000000	75333.333333333333
History	61000.000000000000	62000.000000000000
Music	40000.000000000000	64000.000000000000
Physics	91000.000000000000	65500.000000000000

```
myuniversitydb=# SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER w
                 FROM teacher
                 GROUP BY dept_name
                 WINDOW w AS (ORDER BY dept_name
                              ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
 dept_name  |         avg          |         avg
------------+----------------------+----------------------
 Biology    | 72000.000000000000 | 74666.666666666667
 Comp. Sci. | 77333.333333333333 | 76444.444444444444
 Elec. Eng. | 80000.000000000000 | 80777.777777777778
 Finance    | 85000.000000000000 | 75333.333333333333
 History    | 61000.000000000000 | 62000.000000000000
 Music      | 40000.000000000000 | 64000.000000000000
 Physics    | 91000.000000000000 | 65500.000000000000
(7 rows)
```

A summary of Window functions

preserve original row count (unlike GROUP BY)

operate on a set of rows positioned related to the current row

keep rows with same attribute value using PARTITION BY

order rows within windows using ORDER BY

define which rows to pick using ROWS keyword

essential for analytical and reporting queries

```
[RANKING_FUNCTION | VALUE_FUNCTION | AGGREGATE_FUNCTION]( [arguments] )
    OVER (
        [existing_window_name]
        |
        [
            PARTITION BY expression1 [, expression2, ...]
            [ORDER BY expression [ASC|DESC] [NULLS FIRST|LAST] [, ...]]
            [
                { ROWS | RANGE | GROUPS }
                BETWEEN frame_start AND frame_end
                |
                { ROWS | RANGE | GROUPS } frame_start
            ]
        ]
    ) AS result_column,
```

new idea

AGGREGATE FUNCTIONS

Aggregations

teacher

name	*dept_name*	*salary*
Mozart	Music	40000.00
El Said	History	60000.00
Califieri	History	62000.00
Srinivasan	Comp. Sci.	65000.00
Crick	Biology	72000.00
Katz	Comp. Sci.	75000.00
Singh	Finance	80000.00
Kim	Elec. Eng.	80000.00
Gold	Physics	87000.00
Wu	Finance	90000.00
Brandt	Comp. Sci.	92000.00
Einstein	Physics	95000.00

```
SELECT name, SUM(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary
      ROWS BETWEEN 1 PRECEDING AND 0 FOLLOWING);
```


name	*sum*
Mozart	40000.00
El Said	100000.00
Califieri	122000.00
Srinivasan	127000.00
Crick	137000.00
Katz	147000.00
Singh	155000.00
Kim	160000.00
Gold	167000.00
Wu	177000.00
Brandt	182000.00
Einstein	187000.00

AGGREGATIONS **OVER w**

Apply aggregration to the rows of the window for each row.

AGGREGATIONS

teacher

name	*dept_name*	*salary*
Mozart	Music	40000.00
El Said	History	60000.00
Califieri	History	62000.00
Srinivasan	Comp. Sci.	65000.00
Crick	Biology	72000.00
Katz	Comp. Sci.	75000.00
Singh	Finance	80000.00
Kim	Elec. Eng.	80000.00
Gold	Physics	87000.00
Wu	Finance	90000.00
Brandt	Comp. Sci.	92000.00
Einstein	Physics	95000.00

```
SELECT name, SUM(salary) OVER
                          (ORDER BY salary)
FROM teacher;
```

name	*sum*
Mozart	40000.00
El Said	100000.00
Califieri	162000.00
Srinivasan	227000.00
Crick	299000.00
Katz	374000.00
Singh	534000.00
Kim	534000.00
Gold	621000.00
Wu	711000.00
Brandt	803000.00
Einstein	898000.00

Remember:
Default window when ROWS absent:
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
(ties included)

AGGREGATIONS

name	*dept_name*	*salary*
Mozart	Music	40000.00
El Said	History	60000.00
Califieri	History	62000.00
Srinivasan	Comp. Sci.	65000.00
Crick	Biology	72000.00
Katz	Comp. Sci.	75000.00
Singh	Finance	80000.00
Kim	Elec. Eng.	80000.00
Gold	Physics	87000.00
Wu	Finance	90000.00
Brandt	Comp. Sci.	92000.00
Einstein	Physics	95000.00

```
SELECT name, AVG(salary) OVER
                (ORDER BY salary)
FROM teacher;
```

name	*dept_name*	*avg*
Mozart	Music	40000.000000000000
El Said	History	50000.000000000000
Califieri	History	54000.000000000000
Srinivasan	Comp. Sci.	56750.000000000000
Crick	Biology	59800.000000000000
Katz	Comp. Sci.	62333.333333333333
Singh	Finance	66750.000000000000
Kim	Elec. Eng.	66750.000000000000
Gold	Physics	69000.000000000000
Wu	Finance	71100.000000000000
Brandt	Comp. Sci.	73000.000000000000
Einstein	Physics	74833.333333333333

```
SELECT name, MIN(salary) OVER
                (ORDER BY salary)
FROM teacher;
```

name	*dept_name*	*min*
Mozart	Music	40000.00
El Said	History	40000.00
Califieri	History	40000.00
Srinivasan	Comp. Sci.	40000.00
Crick	Biology	40000.00
Katz	Comp. Sci.	40000.00
Singh	Finance	40000.00
Kim	Elec. Eng.	40000.00
Gold	Physics	40000.00
Wu	Finance	40000.00
Brandt	Comp. Sci.	40000.00
Einstein	Physics	40000.00

```
SELECT name, MAX(salary) OVER
                (ORDER BY salary)
FROM teacher;
```

name	*dept_name*	*max*
Mozart	Music	40000.00
El Said	History	60000.00
Califieri	History	62000.00
Srinivasan	Comp. Sci.	65000.00
Crick	Biology	72000.00
Katz	Comp. Sci.	75000.00
Singh	Finance	80000.00
Kim	Elec. Eng.	80000.00
Gold	Physics	87000.00
Wu	Finance	90000.00
Brandt	Comp. Sci.	92000.00
Einstein	Physics	95000.00

avg **sum** **max** **min** **count**

input must be numbers

new idea

RANKING FUNCTIONS

RANK, DENSE_RANK, ROW_NUMBER, NTILE(n)

RANK

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Give teacher names with salary ranks.

```
SELECT name, RANK() OVER
                (ORDER BY salary ASC)
FROM teacher;
```

name	*rank*
Mozart	1
El Said	2
Califieri	3
Srinivasan	4
Crick	5
Katz	6
Singh	7
Kim	7
Gold	9
Wu	10
Brandt	11
Einstein	12

RANK() OVER
a window function that assigns a rank to each row.
rows with the same values receive the same rank, and the next rank is skipped

RANK

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name, RANK() OVER
             (ORDER BY salary DESC)
FROM teacher;
```

name	*rank*
Einstein	1
Brandt	2
Wu	3
Gold	4
Singh	5
Kim	5
Katz	7
Crick	8
Srinivasan	9
Califieri	10
El Said	11
Mozart	12

```
SELECT name, RANK() OVER
             (ORDER BY dept_name ASC)
FROM teacher;
```

name	*rank*
Crick	1
Srinivasan	2
Katz	2
Brandt	2
Kim	5
Wu	6
Singh	6
El Said	8
Califieri	8
Mozart	10
Gold	11
Einstein	11

```
SELECT name, RANK() OVER
             (ORDER BY dept_name ASC)
FROM teacher
ORDER BY name;
```

name	*rank*
Brandt	2
Califieri	8
Crick	1
Einstein	11
El Said	8
Gold	11
Katz	2
Kim	5
Mozart	10
Singh	6
Srinivasan	2
Wu	6

The two ORDER BYs are completely independent.

DENSE_RANK

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

DENSE_RANK() OVER
a window function that assigns a rank to each row.
assigns rank without gaps for ties

```
SELECT name, DENSE_RANK() OVER
             (ORDER BY dept_name ASC)
FROM teacher;
```

name	*dense_rank*
Crick	1
Srinivasan	2
Katz	2
Brandt	2
Kim	3
Wu	4
Singh	4
El Said	5
Califieri	5
Mozart	6
Gold	7
Einstein	7

```
SELECT name, RANK() OVER
             (ORDER BY dept_name ASC)
FROM teacher;
```

name	*rank*
Crick	1
Srinivasan	2
Katz	2
Brandt	2
Kim	5
Wu	6
Singh	6
El Said	8
Califieri	8
Mozart	10
Gold	11
Einstein	11

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

ROW_NUMBER() OVER
assigns unique sequential numbers

```
SELECT name, ROW_NUMBER() OVER
            (ORDER BY dept_name)
FROM teacher;
```

name	*row_number*
Crick	1
Srinivasan	2
Katz	3
Brandt	4
Kim	5
Wu	6
Singh	7
El Said	8
Califieri	9
Mozart	10
Gold	11
Einstein	12

```
SELECT name, RANK() OVER
            (ORDER BY dept_name)
FROM teacher;
```

name	*rank*
Crick	1
Srinivasan	2
Katz	2
Brandt	2
Kim	5
Wu	6
Singh	6
El Said	8
Califieri	8
Mozart	10
Gold	11
Einstein	11

NTILE(n)

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

NTILE(n) OVER
divide rows into n equal-length blocks

```
SELECT name, NTILE(2) OVER
            (ORDER BY dept_name)
FROM teacher;
```

name	*ntile*
Crick	1
Srinivasan	1
Katz	1
Brandt	1
Kim	1
Wu	1
Singh	2
El Said	2
Califieri	2
Mozart	2
Gold	2
Einstein	2

```
SELECT name, RANK() OVER
            (ORDER BY dept_name)
FROM teacher;
```

name	*rank*
Crick	1
Srinivasan	2
Katz	2
Brandt	2
Kim	5
Wu	6
Singh	6
El Said	8
Califieri	8
Mozart	10
Gold	11
Einstein	11

```
SELECT name, NTILE(5) OVER
            (ORDER BY dept_name)
FROM teacher;
```

name	*ntile*
Crick	1
Srinivasan	1
Katz	1
Brandt	2
Kim	2
Wu	2
Singh	3
El Said	3
Califieri	4
Mozart	4
Gold	5
Einstein	5

Note: Unlike AGGREGATION FUNCTIONS, RANKING FUNCTIONS **ignore** ROWS BETWEEN

teacher

id	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00
76543	Singh	Finance	80000.00
12121	Wu	Finance	90000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
15151	Mozart	Music	40000.00
33456	Gold	Physics	87000.00
22222	Einstein	Physics	95000.00

```
SELECT name, row_number() OVER w
FROM teacher
WINDOW w AS (ORDER BY name
     ROWS BETWEEN 0 PRECEDING AND 0 FOLLOWING);
```

name	*row_number*
Brandt	1
Califieri	2
Crick	3
Einstein	4
El Said	5
Gold	6
Katz	7
Kim	8
Mozart	9
Singh	10
Srinivasan	11
Wu	12

```
SELECT name, rank() OVER w
FROM teacher
WINDOW w AS (ORDER BY salary
     ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

name	*rank*
Mozart	1
El Said	2
Califieri	3
Srinivasan	4
Crick	5
Katz	6
Singh	7
Kim	7
Gold	9
Wu	10
Brandt	11
Einstein	12

new idea

VALUE FUNCTIONS

lag,lead,nth_value

So far: a **window** allows us to do aggregation over ordered rows

new advantage: can also access other row *attribute values*!

```
SELECT name, LAG(name, 2) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary);
```

teacher

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

LAG (attrib, num) OVER w
Get the value of column 'attrib' that is num rows **earlier** in the window w

id	*name*	*lag*
15151	Mozart	
32343	El Said	
58583	Califieri	Mozart
10101	Srinivasan	El Said
76766	Crick	Califieri
45565	Katz	Srinivasan
76543	Singh	Crick
98345	Kim	Katz
33456	Gold	Singh
12121	Wu	Kim
83821	Brandt	Gold
22222	Einstein	Wu

```
SELECT name, LEAD(name, 2) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary);
```

teacher

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

LEAD (attrib, num) OVER w
Get the value of column `attrib` that is num rows **later** in the window w

id	*name*	*lead*
15151	Mozart	Califieri
32343	El Said	Srinivasan
58583	Califieri	Crick
10101	Srinivasan	Katz
76766	Crick	Singh
45565	Katz	Kim
76543	Singh	Gold
98345	Kim	Wu
33456	Gold	Brandt
12121	Wu	Einstein
83821	Brandt	
22222	Einstein	

```
SELECT name, NTH_VALUE(name, 2) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary);
```

NTH_VALUE (attrib, num) OVER w
Get the value of column 'attrib' that is at row number *num* in the window w

id	*name*	*nth_value*
15151	Mozart	
32343	El Said	El Said
58583	Califieri	El Said
10101	Srinivasan	El Said
76766	Crick	El Said
45565	Katz	El Said
76543	Singh	El Said
98345	Kim	El Said
33456	Gold	El Said
12121	Wu	El Said
83821	Brandt	El Said
22222	Einstein	El Said

Mozart missing the value because ?

Default window when ROWS absent:
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
(ties included)

```
SELECT name, NTH_VALUE(name, 2) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary
       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

name	*nth_value*
Mozart	El Said
El Said	El Said
Califieri	El Said
Srinivasan	El Said
Crick	El Said
Katz	El Said
Singh	El Said
Kim	El Said
Gold	El Said
Wu	El Said
Brandt	El Said
Einstein	El Said

NTH_VALUE (attrib, num) OVER w
Get the value of column 'attrib' that is at row number *num* in the window w

For each teacher, give difference from the salary of previous teacher.

```
SELECT name, salary - LAG(salary, 1) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary);
```

teacher

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
10101	Srinivasan	Comp. Sci.	65000.00
76766	Crick	Biology	72000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
33456	Gold	Physics	87000.00
12121	Wu	Finance	90000.00
83821	Brandt	Comp. Sci.	92000.00
22222	Einstein	Physics	95000.00

name	*?column?*
Mozart	
El Said	20000.00
Califieri	2000.00
Srinivasan	3000.00
Crick	7000.00
Katz	3000.00
Singh	5000.00
Kim	0.00
Gold	7000.00
Wu	3000.00
Brandt	2000.00
Einstein	3000.00

new idea

PARTITION BY

PARTITION BY

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Give teacher names with salary rank within same department

```
SELECT name, dept_name, RANK() OVER
                (PARTITION BY dept_name
                 ORDER BY salary)
FROM teacher
ORDER BY dept_name;
```

name	*dept_name*	*rank*
Crick	Biology	1
Srinivasan	Comp. Sci.	1
Katz	Comp. Sci.	2
Brandt	Comp. Sci.	3
Kim	Elec. Eng.	1
Singh	Finance	1
Wu	Finance	2
El Said	History	1
Califieri	History	2
Mozart	Music	1
Gold	Physics	1
Einstein	Physics	2

PARTITION BY dept_name

1. make mini-tables according to dept_name
2. for each row, apply RANK() over its **own mini-table**.

new idea

FRAME CLAUSES

rows

```
SELECT name, dept_name, salary, AVG(salary) OVER
                (PARTITION BY dept_name
                 ORDER BY salary
                 ROWS UNBOUNDED PRECEDING)
FROM teacher;
```



name	*dept_name*	*salary*	*avg*
Crick	Biology	72000.00	72000.000000000000
Srinivasan	Comp. Sci.	65000.00	65000.000000000000
Katz	Comp. Sci.	75000.00	70000.000000000000
Brandt	Comp. Sci.	92000.00	77333.333333333333
Kim	Elec. Eng.	80000.00	80000.000000000000
Singh	Finance	80000.00	80000.000000000000
Wu	Finance	90000.00	85000.000000000000
El Said	History	60000.00	60000.000000000000
Califieri	History	62000.00	61000.000000000000
Mozart	Music	40000.00	40000.000000000000
Gold	Physics	87000.00	87000.000000000000
Einstein	Physics	95000.00	91000.000000000000

```
SELECT course_id, title, credits, SUM(credits) OVER
            (ORDER BY credits
             ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
FROM course;
```



course_id	*title*	*credits*	*sum*
EE-181	Intro. to Digital Systems	3	44
FIN-201	Investment Banking	3	41
HIS-351	World History	3	38
MU-199	Music Video Production	3	35
BIO-399	Computational Biology	3	32
CS-315	Robotics	3	29
CS-319	Image Processing	3	26
CS-347	Database System Concepts	3	23
PHY-101	Physical Principles	4	20
BIO-301	Genetics	4	16
CS-101	Intro. to Computer Science	4	12
CS-190	Game Design	4	8
BIO-101	Intro. to Biology	4	4

```
SELECT name, dept_name, salary, SUM(salary) OVER
                     (PARTITION BY dept_name
                      ORDER BY salary
                      ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM teacher;
```



name	*dept_name*	*salary*	*sum*
Crick	Biology	72000.00	72000.00
Srinivasan	Comp. Sci.	65000.00	140000.00
Katz	Comp. Sci.	75000.00	232000.00
Brandt	Comp. Sci.	92000.00	167000.00
Kim	Elec. Eng.	80000.00	80000.00
Singh	Finance	80000.00	170000.00
Wu	Finance	90000.00	170000.00
El Said	History	60000.00	122000.00
Califieri	History	62000.00	122000.00
Mozart	Music	40000.00	40000.00
Gold	Physics	87000.00	182000.00
Einstein	Physics	95000.00	182000.00

```
SELECT name, salary, dept_name, AVG(salary) OVER w
FROM teacher
WINDOW w AS
 (PARTITION BY dept_name
  ORDER BY salary
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING EXCLUDE CURRENT ROW);
```



name	*salary*	*dept_name*	*avg*
Crick	72000.00	Biology	
Srinivasan	65000.00	Comp. Sci.	83500.000000000000
Katz	75000.00	Comp. Sci.	78500.000000000000
Brandt	92000.00	Comp. Sci.	70000.000000000000
Kim	80000.00	Elec. Eng.	
Singh	80000.00	Finance	90000.000000000000
Wu	90000.00	Finance	80000.000000000000
El Said	60000.00	History	62000.000000000000
Califieri	62000.00	History	60000.000000000000
Mozart	40000.00	Music	
Gold	87000.00	Physics	95000.000000000000
Einstein	95000.00	Physics	87000.000000000000

```
SELECT id, course_id, semester, year,
       COUNT(*) OVER
           (PARTITION BY id
            ORDER BY year, semester
            ROWS UNBOUNDED PRECEDING),
       AVG(COUNT(*)) OVER
           (PARTITION BY id
            ORDER BY year, semester
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
FROM teaches
GROUP BY id, course_id, semester, year;
```

a complicated example

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

id	*course_id*	*semester*	*year*	*count*	*avg*
10101	CS-347	Fall	2009	1	1.0000000000000000000
10101	CS-101	Fall	2009	2	1.0000000000000000000
10101	CS-315	Spring	2010	3	1.0000000000000000000
12121	FIN-201	Spring	2010	1	1.0000000000000000000
15151	MU-199	Spring	2010	1	1.0000000000000000000
22222	PHY-101	Fall	2009	1	1.0000000000000000000
32343	HIS-351	Spring	2010	1	1.0000000000000000000
45565	CS-101	Spring	2010	1	1.0000000000000000000
45565	CS-319	Spring	2010	2	1.0000000000000000000
76766	BIO-101	Summer	2009	1	1.0000000000000000000
76766	BIO-301	Summer	2010	2	1.0000000000000000000
83821	CS-190	Spring	2009	1	2.0000000000000000
83821	CS-319	Spring	2010	2	1.5000000000000000
98345	EE-181	Spring	2009	1	1.0000000000000000000

new idea

FRAME CLAUSES

range

```
SELECT name, salary, COUNT(*) OVER
   (ORDER BY salary
    RANGE BETWEEN 15000 PRECEDING AND 5000 FOLLOWING)
FROM teacher;
```



name	*salary*	*rangesalary*
Mozart	40000.00	1
El Said	60000.00	3
Califieri	62000.00	3
Srinivasan	65000.00	3
Crick	72000.00	5
Katz	75000.00	7
Singh	80000.00	5
Kim	80000.00	5
Gold	87000.00	7
Wu	90000.00	7
Brandt	92000.00	6
Einstein	95000.00	6

RANGE
include rows by value instead of position

RANGE
requires ORDER BY

```
SELECT name, salary, COUNT(*) OVER
   (ORDER BY salary, name
    RANGE BETWEEN 15000 PRECEDING AND 5000 FOLLOWING)
FROM teacher;
```

RANGE
include rows by value instead of position



```
myuniversitydb=# SELECT name, salary, COUNT(*) OVER
myuniversitydb-#    (ORDER BY salary, name
myuniversitydb(#     RANGE BETWEEN 15000 PRECEDING AND 5000 FOLLOWING)
myuniversitydb-# FROM teacher;
ERROR:  RANGE with offset PRECEDING/FOLLOWING requires exactly one ORDER BY column
LINE 2:    (ORDER BY salary, name
           ^
```

RANGE with offset PRECEDING/FOLLOWING requires **exactly one** ORDER BY column.
Adding **name** is impossible **and** unnecessary: RANGE already treats all rows with the same salary as **peers**.

```
SELECT s.dept_name, s.ID, s.tot_cred,
 AVG(tot_cred) OVER
   (PARTITION BY s.dept_name
    ORDER BY tot_cred
    ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING) as local_avg,
 AVG(tot_cred) OVER
    (PARTITION BY s.dept_name
     ORDER BY tot_cred
     RANGE BETWEEN 10 PRECEDING AND 10 FOLLOWING) as range_avg
FROM student s;
```

dept_name	*id*	*tot_cred*	*local_avg*	*range_avg*
Biology	98988	120	120.0000000000000000	120.0000000000000000
Comp. Sci.	12345	32	48.0000000000000000	32.0000000000000000
Comp. Sci.	54321	54	61.5000000000000000	56.0000000000000000
Comp. Sci.	76543	58	61.5000000000000000	56.0000000000000000
Comp. Sci.	00128	102	71.3333333333333333	102.0000000000000000
Elec. Eng.	76653	60	79.0000000000000000	60.0000000000000000
Elec. Eng.	98765	98	79.0000000000000000	98.0000000000000000
Finance	23121	110	110.0000000000000000	110.0000000000000000
History	19991	80	80.0000000000000000	80.0000000000000000
Music	55739	38	38.0000000000000000	38.0000000000000000
Physics	70557	0	34.0000000000000000	0.00000000000000000000
Physics	45678	46	34.0000000000000000	51.0000000000000000
Physics	44553	56	34.0000000000000000	51.0000000000000000

new idea

FRAME CLAUSES

group

```
SELECT name, salary, AVG(salary) OVER w
FROM teacher
WINDOW w AS (ORDER BY salary
             GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

name	*salary*	*avg*
Mozart	40000.00	50000.000000000000
El Said	60000.00	54000.000000000000
Califieri	62000.00	62333.333333333333
Srinivasan	65000.00	66333.333333333333
Crick	72000.00	70666.666666666667
Katz	75000.00	76750.000000000000
Singh	80000.00	80500.000000000000
Kim	80000.00	80500.000000000000
Gold	87000.00	84250.000000000000
Wu	90000.00	89666.666666666667
Brandt	92000.00	92333.333333333333
Einstein	95000.00	93500.000000000000

ROWS ...)

name	*salary*	*avg*
Mozart	40000.00	50000.000000000000
El Said	60000.00	54000.000000000000
Califieri	62000.00	62333.333333333333
Srinivasan	65000.00	66333.333333333333
Crick	72000.00	70666.666666666667
Katz	75000.00	75666.666666666667
Singh	80000.00	78333.333333333333
Kim	80000.00	82333.333333333333
Gold	87000.00	85666.666666666667
Wu	90000.00	89666.666666666667
Brandt	92000.00	92333.333333333333
Einstein	95000.00	93500.000000000000

GROUPS BETWEEN
takes 1 preceeding and 1 following (but more if same **value**)
So can be more rows if several rows have same value

GROUPS
requires ORDER BY

```
SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER
            (ORDER BY dept_name
             GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM teacher
GROUP BY dept_name;
```

dept_name	*avg*	*avg*
Biology	72000.000000000000	74666.666666666667
Comp. Sci.	77333.333333333333	76444.444444444444
Elec. Eng.	80000.000000000000	80777.777777777778
Finance	85000.000000000000	75333.333333333333
History	61000.000000000000	62000.000000000000
Music	40000.000000000000	64000.000000000000
Physics	91000.000000000000	65500.000000000000

same when distinct!

```
SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER
            (ORDER BY dept_name
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM teacher
GROUP BY dept_name;
```

```
SELECT dept_name, COUNT(*), AVG(COUNT(*)) OVER
     (ORDER BY COUNT(*)
      GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM teacher
GROUP BY dept_name;
```



dept_name	*count*	*avg*
Music	1	1.5000000000000000
Biology	1	1.5000000000000000
Elec. Eng.	1	1.5000000000000000
Finance	2	1.7142857142857143
History	2	1.7142857142857143
Physics	2	1.7142857142857143
Comp. Sci.	3	2.2500000000000000

dept_name	*count*	*avg*
Music	1	1.5000000000000000
Biology	1	1.5000000000000000
Elec. Eng.	1	1.5000000000000000
Finance	2	1.7142857142857143
History	2	1.7142857142857143
Physics	2	1.7142857142857143
Comp. Sci.	3	2.2500000000000000

ROWS / RANGE / GROUPS BETWEEN

Before

UNBOUNDED PRECEDING

n PRECEDING

CURRENT ROW

After

UNBOUNDED FOLLOWING

n FOLLOWING

CURRENT ROW

new idea

WINDOW FUNCTIONS

examples

```
SELECT dept_name, AVG(salary) OVER w
FROM teacher
GROUP BY dept_name
WINDOW w AS (GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING);
```

Error: GROUPS requires ORDER BY.



```
myuniversitydb=# SELECT dept_name, AVG(salary) OVER w
                 FROM teacher
                 GROUP BY dept_name
                 WINDOW w AS (GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING) ;
ERROR:  GROUPS mode requires an ORDER BY clause
LINE 4:                  WINDOW w AS (GROUPS BETWEEN 1 PRECEDING AND...
                                     ^
```

```
SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER w
FROM teacher
GROUP BY dept_name
WINDOW w AS (ORDER BY name
                GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING) ;
```

Error: after GROUP BY dept_name,
the column name no longer exists per row

```
myuniversitydb=# SELECT dept_name, AVG(salary), AVG(AVG(salary)) OVER w
                 FROM teacher
                 GROUP BY dept_name
                 WINDOW w AS (ORDER BY name
                                 GROUPS BETWEEN 1 PRECEDING AND 1 FOLLOWING) ;
ERROR:  column "teacher.name" must appear in the GROUP BY clause or be used in an aggregate function
LINE 4:                 WINDOW w AS (ORDER BY name
                                     ^
```

```
SELECT d.dept_name, d.building, d.budget,
  COUNT(*) OVER w_dept_count as db,
  AVG(d.budget) OVER w_building_avg as avg_building_budget,
  d.budget-AVG(d.budget) OVER w_building_avg_range as diff_from_nearby,
  PERCENT_RANK() OVER w_dept_order as budget_percentile
FROM department AS d
WINDOW w_base AS (PARTITION BY d.building),
       w_dept_count AS (w_base),
       w_building_avg AS (w_base),
       w_building_avg_range AS (w_base ORDER BY d.budget
                RANGE BETWEEN 10000 PRECEDING AND 10000 FOLLOWING),
       w_dept_order AS (ORDER BY d.budget)
ORDER BY d.building, d.budget;
```

dept_name	*building*	*budget*	*db*	*avg_building_budget*	*diff_from_nearby*	*budget_percentile*
Music	Packard	80000.00	1	80000.000000000000	0.000000000000	0.3333333333333333
History	Painter	50000.00	2	85000.000000000000	0.000000000000	0
Finance	Painter	120000.00	2	85000.000000000000	0.000000000000	1
Elec. Eng.	Taylor	85000.00	2	92500.000000000000	0.000000000000	0.5
Comp. Sci.	Taylor	100000.00	2	92500.000000000000	0.000000000000	0.8333333333333334
Physics	Watson	70000.00	2	80000.000000000000	0.000000000000	0.16666666666666666
Biology	Watson	90000.00	2	80000.000000000000	0.000000000000	0.6666666666666666

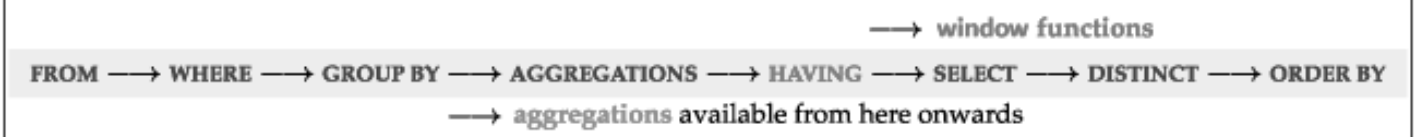
```sql
SELECT t.name, t.dept_name, t.salary, d.building,
    AVG(t.salary) OVER w_dept as avg_dept_salary,
    AVG(t.salary) OVER w_building as avg_building_salary,
    RANK() OVER w_dept_order as salary_rank_in_dept
FROM teacher t JOIN department d ON t.dept_name = d.dept_name
WINDOW w_dept AS (PARTITION BY t.dept_name),
       w_building AS (PARTITION BY d.building),
       w_dept_order AS (PARTITION BY t.dept_name
                        ORDER BY t.salary DESC)
ORDER BY t.dept_name, t.salary DESC;
```

name	*dept_name*	*salary*	*building*	*avg_dept_salary*	*avg_building_salary*	*salary_rank_in_dept*
Crick	Biology	72000.00	Watson	72000.000000000000	84666.666666666667	1
Brandt	Comp. Sci.	92000.00	Taylor	77333.333333333333	78000.000000000000	1
Katz	Comp. Sci.	75000.00	Taylor	77333.333333333333	78000.000000000000	2
Srinivasan	Comp. Sci.	65000.00	Taylor	77333.333333333333	78000.000000000000	3
Kim	Elec. Eng.	80000.00	Taylor	80000.000000000000	78000.000000000000	1
Wu	Finance	90000.00	Painter	85000.000000000000	73000.000000000000	1
Singh	Finance	80000.00	Painter	85000.000000000000	73000.000000000000	2
Califieri	History	62000.00	Painter	61000.000000000000	73000.000000000000	1
El Said	History	60000.00	Painter	61000.000000000000	73000.000000000000	2
Mozart	Music	40000.00	Packard	40000.000000000000	40000.000000000000	1
Einstein	Physics	95000.00	Watson	91000.000000000000	84666.666666666667	1
Gold	Physics	87000.00	Watson	91000.000000000000	84666.666666666667	2

```sql
SELECT s.ID, s.name, t.year, t.semester, c.credits,
    SUM(c.credits) OVER w_running as running_total,
    AVG(c.credits) OVER w_moving as moving_avg_3_courses
FROM (student s JOIN takes t ON s.ID = t.ID)
     JOIN course c ON t.course_id = c.course_id
WHERE t.grade IS NOT NULL AND t.grade != 'F'
WINDOW w_running AS (PARTITION BY s.ID
                     ORDER BY t.year, t.semester),
       w_moving AS (w_running ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY s.ID, t.year, t.semester;
```

id	*name*	*year*	*semester*	*credits*	*running_total*	*moving_avg_3_courses*
00128	Zhang	2009	Fall	4	7	4.0000000000000000
00128	Zhang	2009	Fall	3	7	3.5000000000000000
12345	Shankar	2009	Fall	3	7	3.0000000000000000
12345	Shankar	2009	Fall	4	7	3.5000000000000000
12345	Shankar	2009	Spring	4	11	3.6666666666666667
12345	Shankar	2010	Spring	3	14	3.6666666666666667
19991	Brandt	2010	Spring	3	3	3.0000000000000000
23121	Chavez	2010	Spring	3	3	3.0000000000000000
44553	Peltier	2009	Fall	4	4	4.0000000000000000
45678	Levy	2010	Spring	3	7	3.0000000000000000
45678	Levy	2010	Spring	4	7	3.5000000000000000
54321	Williams	2009	Fall	4	4	4.0000000000000000
54321	Williams	2009	Spring	4	8	4.0000000000000000
55739	Sanchez	2010	Spring	3	3	3.0000000000000000
76543	Brown	2009	Fall	4	4	4.0000000000000000
76543	Brown	2010	Spring	3	7	3.5000000000000000
76653	Aoi	2009	Spring	3	3	3.0000000000000000
98765	Bourikas	2009	Fall	4	4	4.0000000000000000
98765	Bourikas	2010	Spring	3	7	3.5000000000000000
98988	Tanaka	2009	Summer	4	4	4.0000000000000000

Important things to note



Window functions see the dataset after WHERE, GROUP BY, and HAVING are done.

They add columns, they don't reduce the number of rows

Because window functions execute in the SELECT phase, you can't reference their results in earlier clauses like WHERE, GROUP BY or HAVING.
You **can** use them in ORDER BY, or wrap the query in a subquery.

They preserve individual rows—even when used with aggregates, window functions maintain the original row structure rather than collapsing rows like GROUP BY.